



Szoftvertechnológia

Folytonos integráció és kiadás
Continuous integration & delivery

Dr. Cserép Máté
ELTE Informatikai Kar
2025.

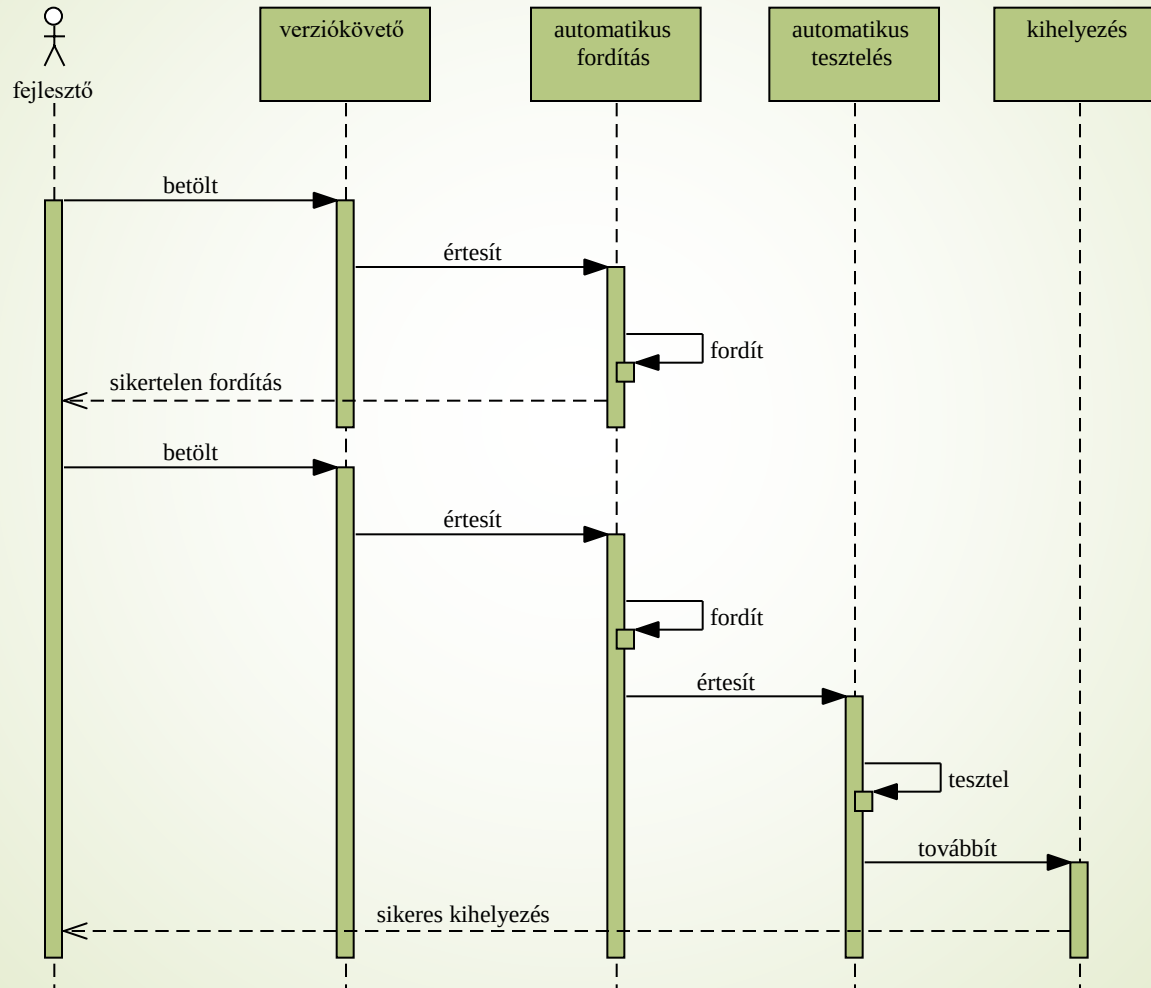
Folyamatos integráció

- A *folytonos integráció* (*continuous integration, CI*) egy olyan gyakorlati módszer, amely lehetővé teszi a programkódok ellenőrzésének és tesztelésének felgyorsítását
 - célja a lehetséges hibák, integrációs problémák azonnali, automatizált kiszűrése, visszajelzés a fejlesztőnek
 - a programkódok verziókezelő rendszer segítségével egy központi tárhelyre kerülnek, naponta többször
 - a tárhely tartalma minden módosítást követően automatikusan fordításra kerül (*build automation*), a fordítással pedig a lekódolt tesztek is végrehajtnak
 - az így ellenőrzött kódot további tesztelés követheti

Folyamatos teljesítés

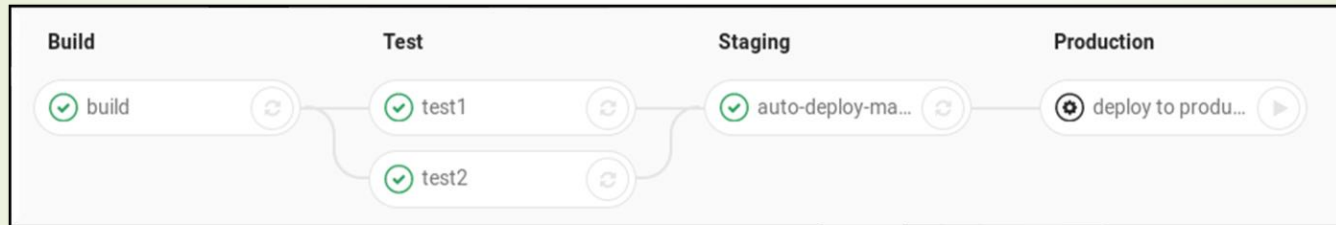
- Az agilis szoftverfejlesztés (*agile software development*) célja a gyors alkalmazásfejlesztés megvalósítása, inkrementális alapon
 - a szoftver folyamatos fejlesztés és kiadás alatt áll (*continuous delivery*), a sebesség állandó, a változtatások minden lépésben beépíthetők (*welcome changes*)
 - a működő szoftver az előrehaladás mérőeszköze, előtérben az egyszerűség, ugyanakkor folyamatos odafigyelés a megfelelő tervezésre, optimalizációra
 - a fejlesztést általában önszervező, kis csapatok végzik, megosztott felelősséggel, folytonos interakcióval, gyors visszajelzésekkel
 - a folyamatos kiadások automatizálhatók, ekkor *continuous deployment*-ről beszélünk

Folyamatos integráció és teljesítés



Feladatok

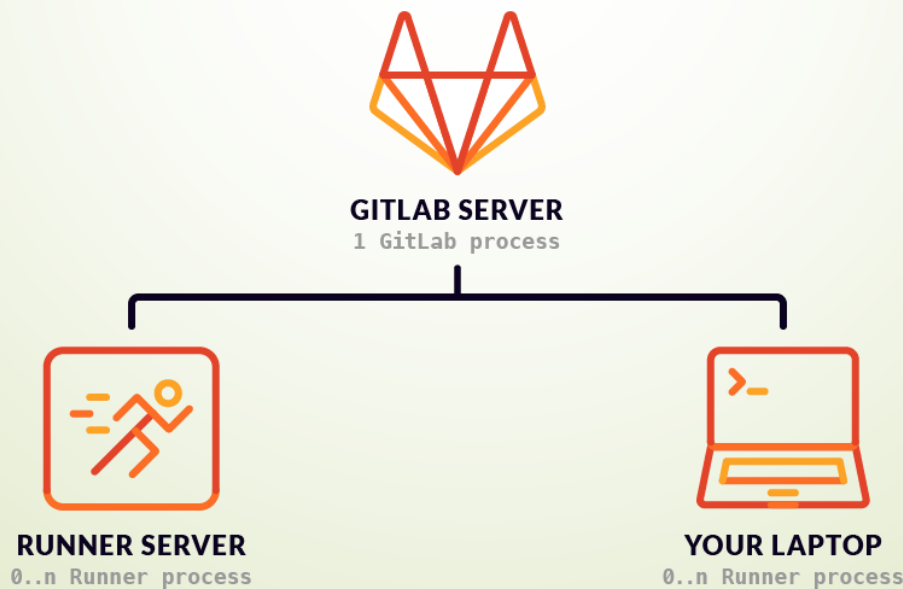
- A folyamatos integráció és teljesítés lépéseit egymásra épülő feladatok (*jobs*) láncolataként (*pipelines*) definiálhatjuk



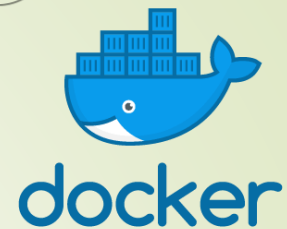
Pipelines Jobs Environments Cycle Analytics					
All 16831	Running 1	Branches	Tags	Run pipeline CI Lint	
Status	Pipeline	Commit	Stages		
running	#6453892 by [user] latest	3df39dd6 add data durability to Alex	✓ 🔄	▶ ⬇ ⬆	
passed	#6453883 by [user] latest	d0f228af Filled in content	✓ ✓	▶ ⬇ ⬆	
passed	#6453817 by [user] latest	06a01e18 De Wet Geo Expert	✓ ✓	▶ ⬇ ⬆	
passed	#6453004 by [user] latest	0c7954e5 Update index.html.md	✓ ✓	⬇ ⬆	
				00:08:15	4 minutes ago
				00:08:50	8 minutes ago
				00:08:33	59 minutes ago

GitLab Runners

- A GitLab rendelkezik integrált, saját megoldással a folyamatos integráció és teljesítés támogatására
 - a feladatokat (*jobs*) a GitLab szervertől független ún. *GitLab Runner* példányok hajtják végre
 - Shell, SSH, VirtualBox, Docker, Kubernetes, stb.
- a *runnerek* egyedileg konfigurálhatóak, lehetnek megosztottak vagy projekthez rendelvek

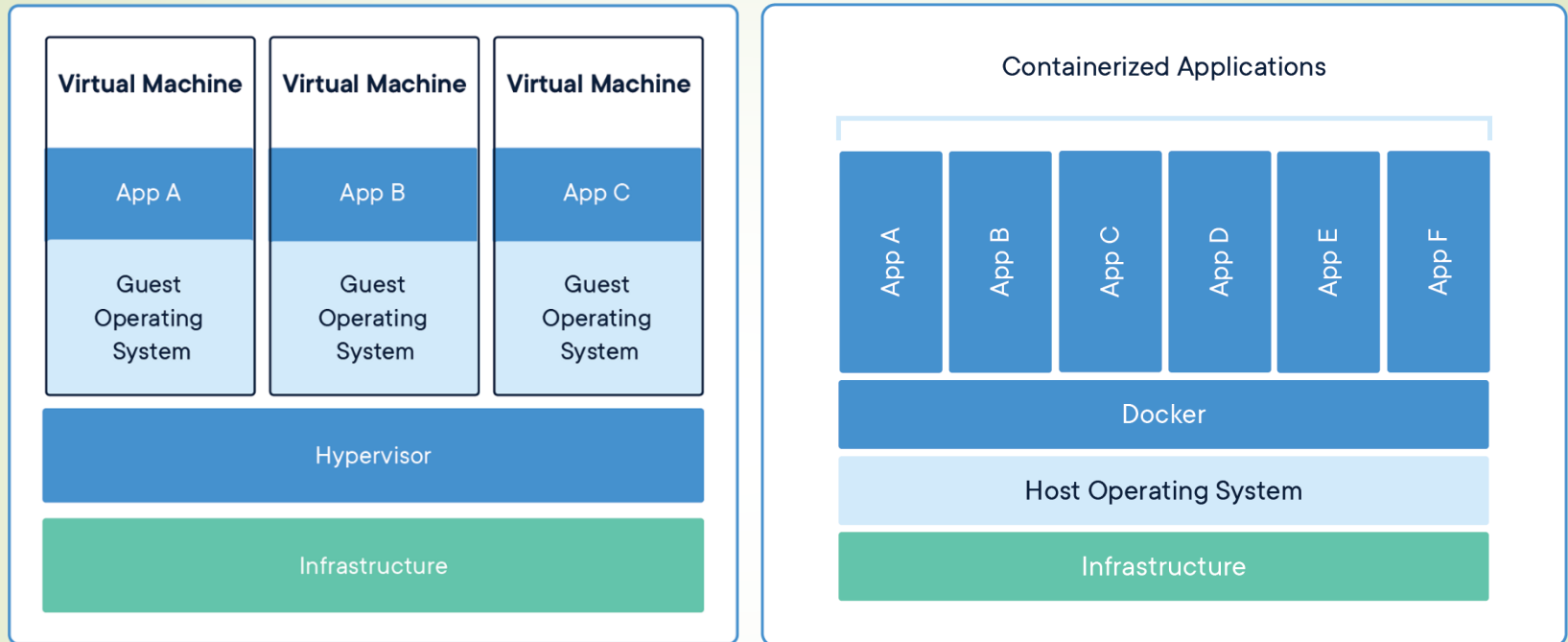


Docker



- A folyamatos integrációt egy izolált, reprodukálható környezetben érdemes végezni
- A Docker napjainkban a legelterjedtebb *container framework*
 - a *container* hasonlít a virtuális gépekhez (VM) olyan tekintetben, hogy egy teljesen elkülönített, virtualizált környezett biztosít, amelynek a gazdaszámítógép szolgáltat erőforrásokat
 - a fő különbség a *containerek* és a virtuális gépek között, hogy minden *container* osztozik a gazda kerneljén a többi *containerrel*, a virtualizált hardver és az OS nem része, csupán az alkalmazásunkhoz kötődő könyvtárak, binárisok és a felhasználói terület
 - a *containerek* ezáltal nagyságrendekkel kisebb *overheaddel* bírnak a VM-ekhez képest, így könnyebb súlyú megoldást nyújtanak a virtualizációra

Virtuális gépek és *container*ek

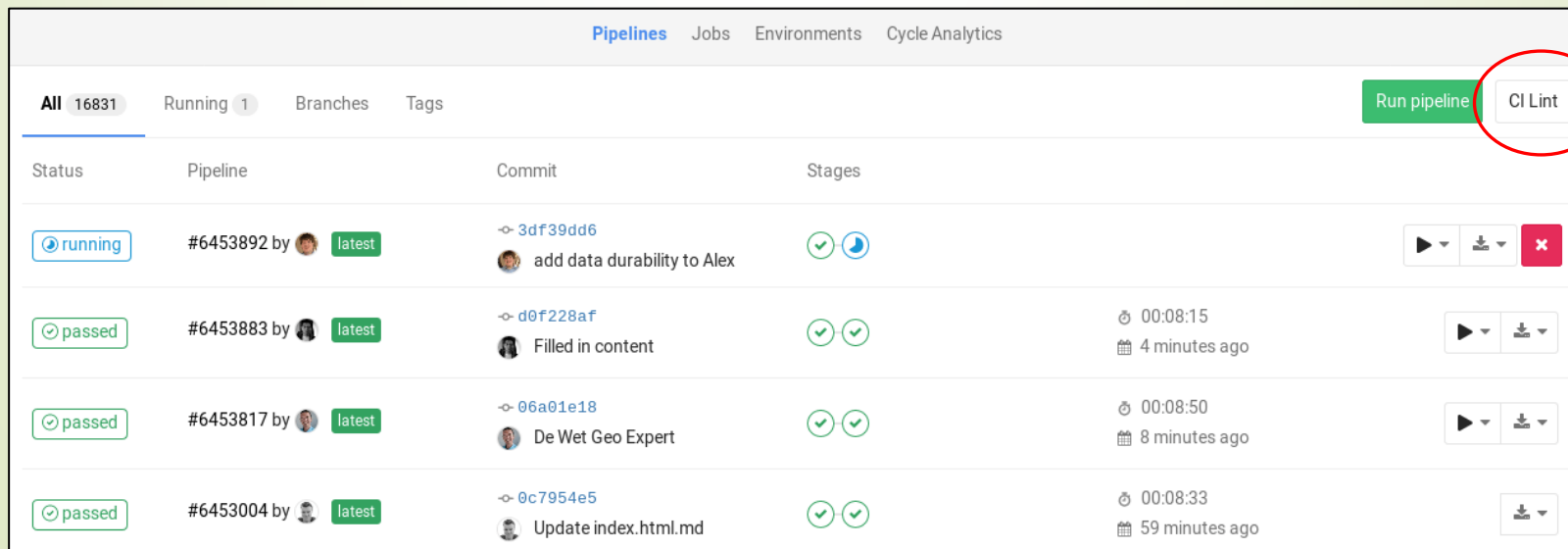


Forrás: docker.com

- a *container framework*ök lehetőséget adnak hordozható alkalmazások létrehozására és menedzselésére
- az alkalmazások modularizálhatóak és skálázhatóak, a komponensek külön *container*ben futhatnak

GitLab CI/CD

- A folyamatos integráció konfigurációját a `.gitlab-ci.yml` fájl tartalmazza, YAML formátumban
 - a YAML (*YAML Ain't Markup Language*) egy emberi szemmel könnye(bbe)n olvasható strukturált leíró nyelv
 - <https://yaml.org/spec/1.2/spec.html>
- A GitLab webes felületén elérhető egy CI Lint funkció a formátum validálására beküldés előtt



The screenshot shows the GitLab Pipelines page. At the top, there are tabs for 'Pipelines', 'Jobs', 'Environments', and 'Cycle Analytics'. Below the tabs, there are filters for 'All 16831', 'Running 1', 'Branches', and 'Tags'. On the right, there are two buttons: 'Run pipeline' and 'CI Lint'. The 'CI Lint' button is circled in red. Below the buttons, there is a table with columns: 'Status', 'Pipeline', 'Commit', and 'Stages'. The table lists four pipelines. The first pipeline is 'running' and has a 'CI Lint' button next to it. The other three pipelines are 'passed'.

Status	Pipeline	Commit	Stages
running	#6453892 by [user] latest	3df39dd6 add data durability to Alex	[check] [refresh]
passed	#6453883 by [user] latest	d0f228af Filled in content	[check] [check]
passed	#6453817 by [user] latest	06a01e18 De Wet Geo Expert	[check] [check]
passed	#6453004 by [user] latest	0c7954e5 Update index.html.md	[check] [check]

YAML szintaxis

► Példa YAML kód:

```
name: Gipsz Jakab # kulcs-érték párok
age: 42
details: # beágyazott kollekció
  givenname: Jakab
  familyname: Gipsz
  birthyear: 1978
languages: # értékek listája (tömb)
  - Hungarian
  - English
  - German
# több soros szöveg
intro_multi: |
  multiple line
  introduction
intro_single: >
  single line
  introduction
```

GitLab CI/CD: jobs

- ▶ A `.gitlab-ci.yml` fájlban feladatokat (*jobs*) definiálhatunk, amelyekben megadhatjuk milyen utasításokat kell végrehajtaniuk (`script`).
- ▶ Pl.:

build_program:

script:

- `apt-get update -qq`
- `apt-get install -yqq openjdk-11-jdk ant`
- `ant compile`
- `ant jar`

GitLab CI/CD: multiple jobs

- Több feladat is definiálható, továbbá megadható egy globális `before_script` elem is, amelyet minden *job* előtt végre kell hajtani. (Felüldefiniálható az egyes feladatokban.)

`before_script:`

- `apt-get update -qq`
- `apt-get install -yqq openjdk-11-jdk ant junit`

`build_program:`

`script:`

- `ant compile`
- `ant jar`

`test_program:`

`script:`

- `ant compile-test`
- `ant test`

GitLab CI/CD: stages

- A folyamatos integráció feladatait egymást követő szakaszokra (*stages*) oszthatjuk
 - alapértelmezetten 3 *stage* van: *build*, *test*, *deploy*
 - ez tetszőlegesen felüldefiniálhatjuk
- ```
stages:
 - lint
 - build
```
- Egy *stage* feladatai egymástól függetlenül párhuzamosítva végrehajthatóak (több *runner* bevonásával)
    - a *stagek* egymásra épülnek, amennyiben egy *stage* valamely feladata hibával zárul, a rá épülő *stagek* nem kerülnek végrehajtásra

## GitLab CI/CD: stages

- A folyamat *stage*kre osztása:

```
before_script:
```

- apt-get update -qq
- apt-get install -yqq openjdk-11-jdk ant junit

```
build_program:
```

```
stage: build
```

```
script:
```

- ant compile
- ant jar

```
test_program:
```

```
stage: test
```

```
script:
```

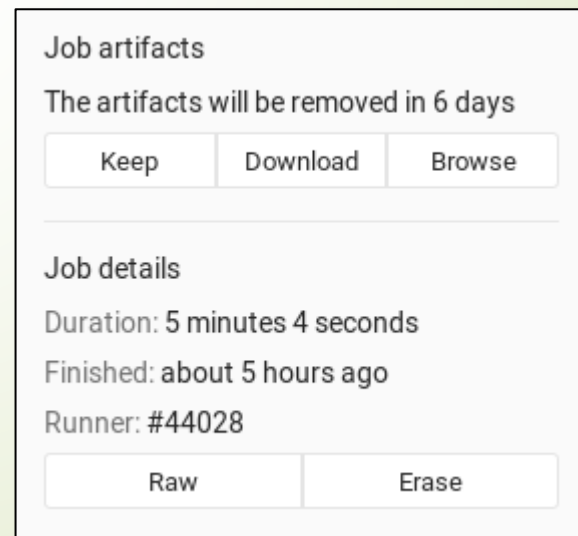
- ant compile-test
- ant test

## GitLab CI/CD: artifacts

- A CI feladatok részeként előállított bináris vagy egyéb állományokat megőrizhetjük (*artifact*)

```
pdf:
 script:
 - pdftex paper.tex
 artifacts:
 paths:
 - paper.pdf
 expire_in: 1 month
```

- Az *artifactok* a GitLab webes felületéről könnyen letölthetőek





## GitLab CI/CD: artifacts

### ► *Artifactok* definiálása:

```
before_script:
 - apt-get update -qq
 - apt-get install -yqq openjdk-11-jdk ant

build_program:
 stage: build
 script:
 - ant compile
 - ant jar
 artifacts:
 paths:
 - dist/program.jar
 expire_in: 1 week
```

## GitLab CI/CD: dependencies

- Az egymástól függő programok (modulok) ellenőrzése könnyen redundáns végrehajtáshoz vezethet:

```
before_script: ...
```

```
build_program_A:
 stage: build
 script:
 - cd module_A
 - ant jar
```

```
build_program_B: # függ program_A.jar állománytól
 stage: build
 script:
 - cd module_A
 - ant jar
 - cd ..
 - cd module_B
 - ant jar
```

## GitLab CI/CD: dependencies

► *Artifactok átadása jobok között:*

```
before_script: ...
```

```
build_program_A:
```

```
 stage: build
```

```
 script:
```

- cd module\_A
- ant jar

```
 artifacts:
```

```
 paths:
```

- module\_A/dist/program\_A.jar

```
build_program_B: # függ program_A.jar állománytól
```

```
 stage: build
```

```
 script:
```

- cd module\_B
- ant jar

```
 dependencies:
```

- build\_program\_A

## Docker images

- *Docker container* alapú GitLab Runner esetén megadhatjuk melyik *docker image*-ből kívánunk kiindulni:

image: ubuntu:22.04 *image név*

- *Docker image*-t egy *Docker registry*ből kérhetünk:

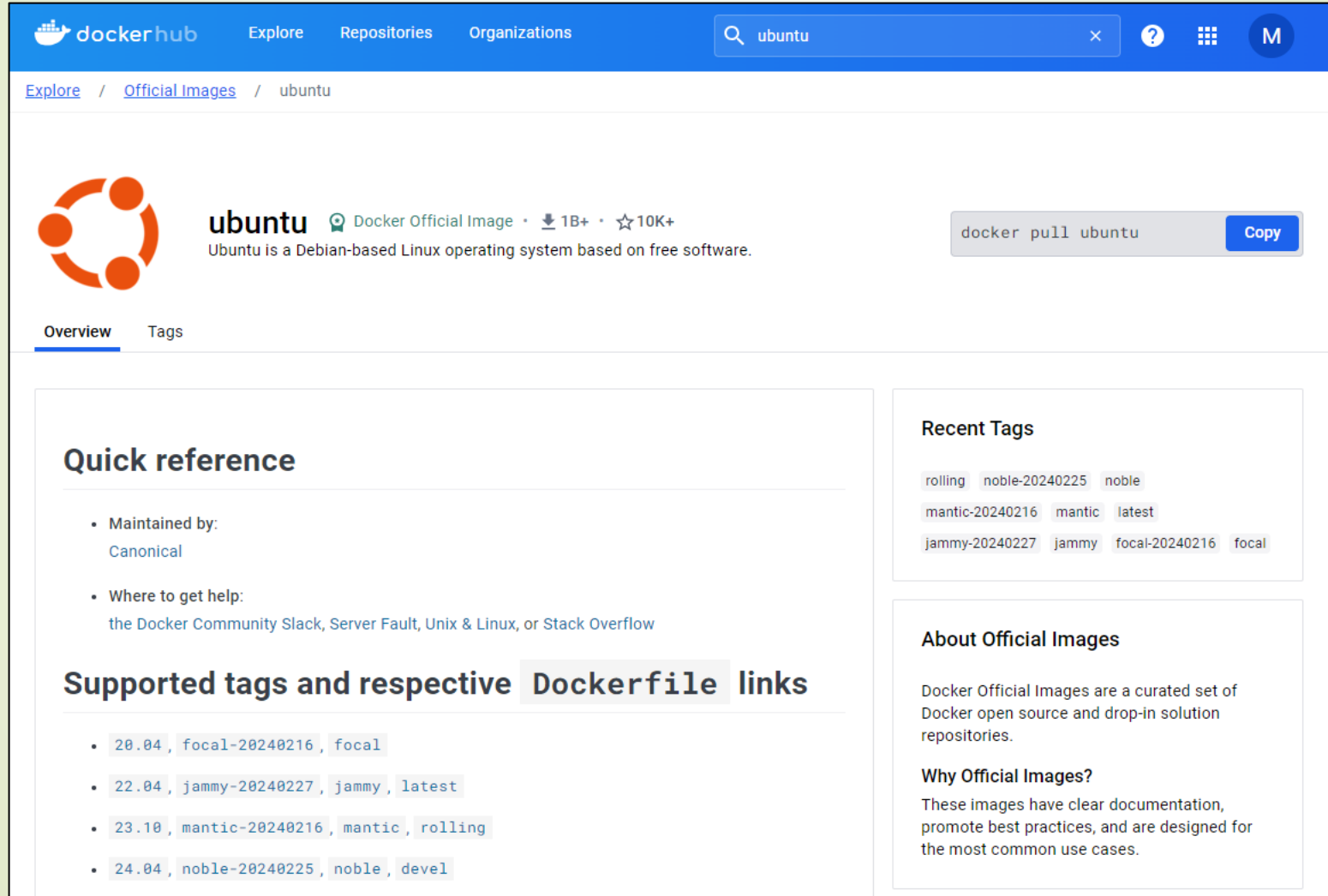
- Alapértelmezetten a publikus Docker Hub-ot használjuk, ahová saját image is feltölthető: <https://hub.docker.com/>

- Használható privát *Docker registry* is (pl. vállalati környezet)

image: mycompany.com:5000/custom:latest

- Ha nem adjuk meg, akkor a runner konfigurációja adja meg a használandó image-t
  - a szofttech.inf.elte.hu runnerjei az ubuntu:22.04 imagere vannak konfigurálva
  - A Windows runnerekhez az alapértelmezett image:  
mcr.microsoft.com/windows/servercore:1809

## Docker Hub



The screenshot shows the Docker Hub interface for the 'ubuntu' image. The top navigation bar includes 'dockerhub', 'Explore', 'Repositories', and 'Organizations'. A search bar contains 'ubuntu'. The breadcrumb trail is 'Explore / Official Images / ubuntu'. The main section features the Ubuntu logo, the name 'ubuntu', and the text 'Docker Official Image' with download and star counts. A 'docker pull ubuntu' command is shown with a 'Copy' button. Below this are tabs for 'Overview' and 'Tags'. The 'Overview' tab is active, showing a 'Quick reference' section with links to Canonical and Docker community resources. A 'Supported tags and respective Dockerfile links' section lists various tags like '20.04', 'focal-20240216', 'focal', etc. A 'Recent Tags' section on the right lists tags like 'rolling', 'noble-20240225', 'noble', etc. An 'About Official Images' section explains that these are curated sets of Docker open source and drop-in solution repositories. A 'Why Official Images?' section states that these images have clear documentation, promote best practices, and are designed for the most common use cases.

**ubuntu** Docker Official Image • 1B+ • 10K+

Ubuntu is a Debian-based Linux operating system based on free software.

`docker pull ubuntu` [Copy](#)

[Overview](#) [Tags](#)

### Quick reference

- Maintained by:  
[Canonical](#)
- Where to get help:  
[the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

### Supported tags and respective Dockerfile links

- `20.04`, `focal-20240216`, `focal`
- `22.04`, `jammy-20240227`, `jammy`, `latest`
- `23.10`, `mantic-20240216`, `mantic`, `rolling`
- `24.04`, `noble-20240225`, `noble`, `devel`

### Recent Tags

`rolling` `noble-20240225` `noble`  
`mantic-20240216` `mantic` `latest`  
`jammy-20240227` `jammy` `focal-20240216` `focal`

### About Official Images

Docker Official Images are a curated set of Docker open source and drop-in solution repositories.

### Why Official Images?

These images have clear documentation, promote best practices, and are designed for the most common use cases.

## GitLab CI/CD

- További lehetőségek (teljesség igénye nélkül):
  - `only`, `except`: CI jobok végrehajtásának feltételhez kötése (például csak a *master* branch-en futtatni)
  - `when`: CI jobok végrehajtásának feltételhez kötése (manuális vs. automatikus végrehajtás)

```
image: maven:latest
build_program:
 stage: build
 script:
 - mvn compile
test_program:
 stage: test
 script:
 - mvn test
deploy_program:
 stage: deploy
 script:
 - mvn deploy
only:
 - master
when: manual
```

## GitLab CI/CD

- ▶ `rules`: feltételes végrehajtás haladóbb konfigurációja (`only` és `except` szabályokhoz képest)
  - ▶ `only`, `except` kulcsszavak már elavultak (*deprecated*), újabb fejlesztéseket már nem kapnak
  - ▶ pl. végrehajtás kizárólag a *master* fejlesztési ágon:  

```
rules:
 - if: $CI_COMMIT_BRANCH == "master"
```
  - ▶ pl. végrehajtás csak a *Dockerfile* változása esetén:  

```
rules:
 - changes:
 - Dockerfile
```
- ▶ ezen kondíciók tetszőlegesen kombinálhatóak is



## GitLab CI/CD

- ▀ `variables`: változók definiálása.  
A futtató környezetre számos változó már előre definiált:  
<https://docs.gitlab.com/ee/ci/variables/>
- ▀ `services`: szolgáltatások (pl. adatbázis motor) külön Docker containerben futtatása (*docker-compose*)

```
services:
```

```
 - mysql:latest
```

```
variables:
```

```
 MYSQL_DATABASE: my_db
```

```
 MYSQL_USER: my_user
```

```
 MYSQL_PASSWORD: very_secret_password
```

- ▀ a hosznév `mysql` lesz (`alias` opcióval megadható más)

## GitLab CI/CD

➤ `cache`: fájlok, könyvtárak (tipikusan függőségek) megőrzése CI jobok és pipelineok között

➤ `pl.:`  
`cache:`  
`paths:`  
`- vendor/`

➤ a cache kiüríthető manuálisan:

All 13

Pending 0

Running 0

Finished 13

Branches

Tags

Run Pipeline

Clear runner caches

CI Lint

Status

Pipeline

Commit

Stages

passed

#16189953 by

latest

master 0e5852e6  
 Update .gitlab-ci.yml

✓

✓

✓

✓

✓

»

00:09:52  
 3 days ago

▶

⌵

↺

⌵

passed

#16058118 by

master 12631789  
 Update .gitlab-ci.yml

✓

✓

✓

»

00:06:23  
 a week ago

▶

⌵

↺

⌵

failed

#16055644 by

feature-branch f19b0131  
 Update .gitlab-ci.yml

✓

✓

✓

!

✖

»

00:09:44  
 a week ago

▶

⌵

↺

⌵

## GitLab CI/CD

- Cacheléskor egy kulcs (`key`) is megadható, amellyel pl. CI *job*-onként vagy *branch*-enként külön cache használható.
- Pl. egy Maven alkalmazás esetén megadható a letöltendő csomagok *cachelése*:

`variables:`

```
MAVEN_OPTS: "-Dmaven.repo.local=
 $CI_PROJECT_DIR/.m2/repository"
```

`cache:`

```
Külön cache jobonként és branchenként
```

```
key: "CI_JOB_NAME-CI_COMMIT_REF_SLUG"
```

```
paths:
```

```
- .m2/repository
```

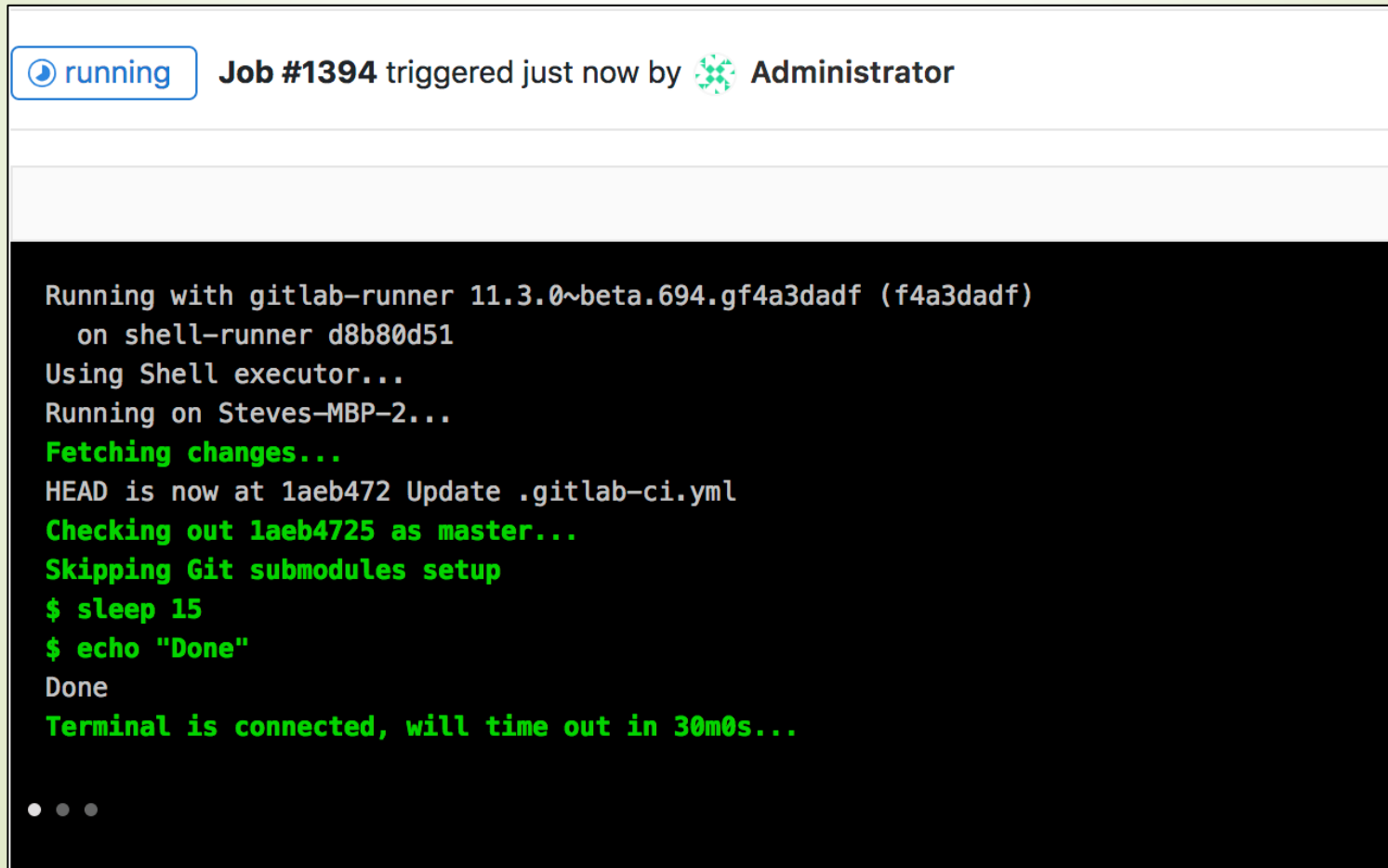
```
build_program:
```

```
script:
```

```
- mvn compile
```

## GitLab CI/CD: terminals

- A folyamatos integráció feladatainak végrehajtását egy online terminál ablakon keresztül követhetjük a GitLab webes felületén



The screenshot shows a GitLab CI/CD job interface. At the top, a status bar indicates the job is 'running' (with a play icon) and 'Job #1394 triggered just now by Administrator' (with a user icon). Below this is a large terminal window with a black background and white text. The terminal output shows the job running on a shell-runner, fetching changes, checking out a specific commit, skipping submodule setup, and executing a sleep command followed by an echo command. The terminal is currently connected and will time out in 30 minutes.

```
Running with gitlab-runner 11.3.0~beta.694.gf4a3dadf (f4a3dadf)
 on shell-runner d8b80d51
Using Shell executor...
Running on Steves-MBP-2...
Fetching changes...
HEAD is now at 1aeb472 Update .gitlab-ci.yml
Checking out 1aeb4725 as master...
Skipping Git submodules setup
$ sleep 15
$ echo "Done"
Done
Terminal is connected, will time out in 30m0s...
```

## GitLab CI/CD: példa projekt

- Hálózati Pacman játék Java implementációval:

<https://szofttech.inf.elte.hu/mate/pacman-java/>

```
1 image: openjdk:11
2
3 stages:
4 - build
5 - test
6
7 before_script:
8 - apt-get update -yqq
9 - apt-get install -yqq ant junit4
10
11 # Build
12 build_game:
13 stage: build
14 script:
15 - ant compile
16 - ant jar
17
18 # Test
19 test_model:
20 stage: test
21 script:
22 - >
23 ant test
24 -Dlibs.junit_4.classpath=/usr/share/java/junit4.jar|
25 -Dlibs.hamcrest.classpath=/usr/share/java/hamcrest-core.jar
26
```

- Ant build rendszerrel: ant fejlesztési ág

## GitLab CI/CD: példa projekt

- Maven build rendszerrel: `maven` fejlesztési ág

```
1 image: maven:latest
2
3 variables:
4 MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository"
5
6 cache:
7 key: maven-build
8 paths:
9 - .m2/repository
10
11 build:
12 stage: build
13 script:
14 - mvn compile
15 test:
16 stage: test
17 script:
18 - mvn test
```

- A cache egy kulccsal azonosítható, így pl. szabályozható, hogy mely fejlesztési ágak között kerüljön megosztásra.

## GitLab CI/CD: példa projekt

- Gradle build rendszerrel: `gradle` fejlesztési ág

```
1 image: gradle:latest
2
3 variables:
4 GRADLE_USER_HOME: "$CI_PROJECT_DIR/.gradle"
5
6 # Cache and keep downloaded dependencies between CI pipelines
7 cache:
8 key: gradle-build
9 paths:
10 - .gradle
11
12 build:
13 stage: build
14 script:
15 - gradle assemble
16
17 test:
18 stage: test
19 script:
20 # build also executes test
21 - gradle build
22
```

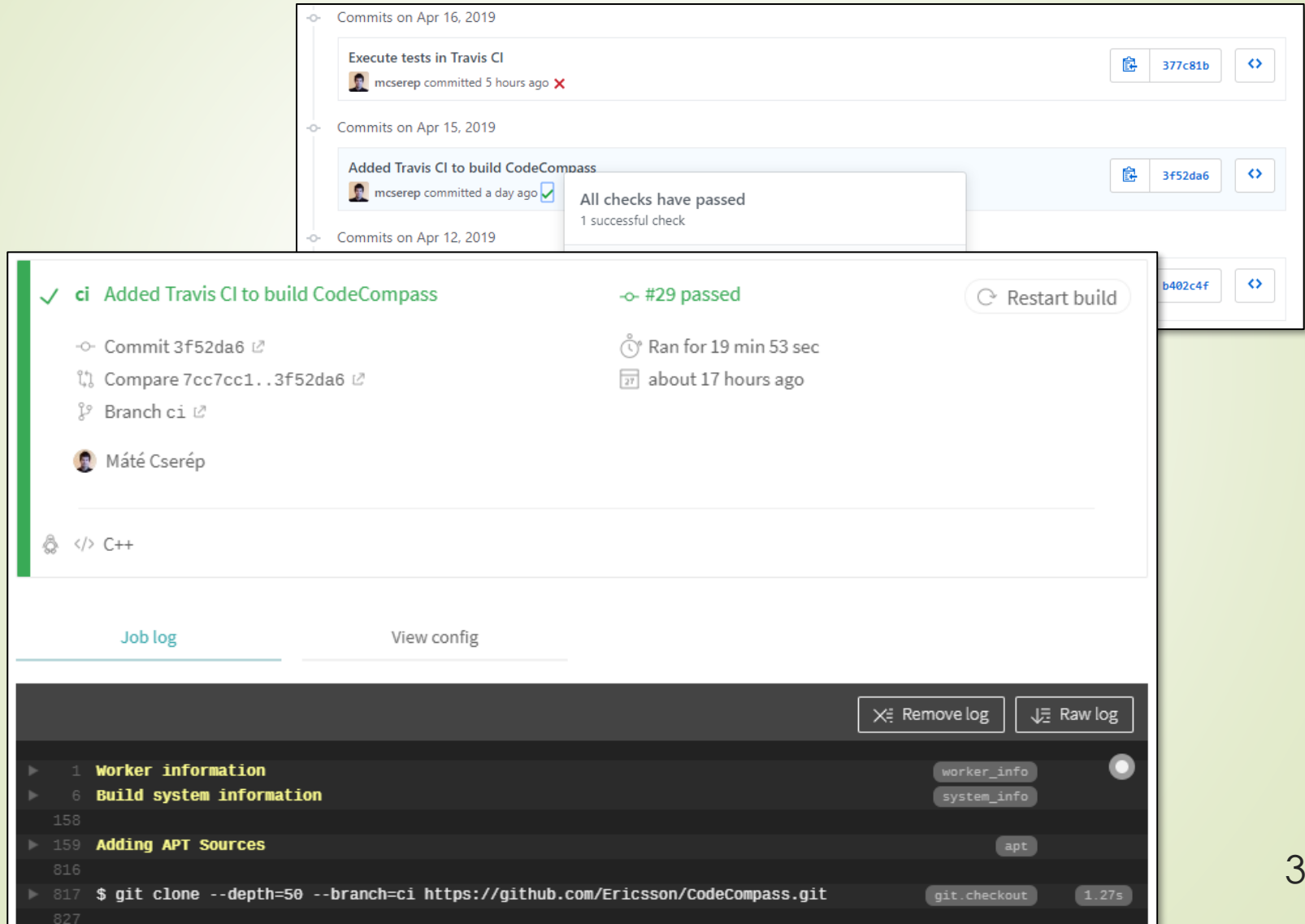


## Travis CI



- Folyamatos integrációs szolgáltatás GitHub projektekhez
  - Korábban népszerű, nyílt forráskódú projektekhez Travis CI ingyenesen használható, de 2021 óta már nem.
  - <https://travis-ci.com/>
- Támogatja a Linux, a Windows és a macOS operációs rendszereket, több előkészített környezettel (*image*)
  - A környezetet a használt programozási nyelvhez állíthatjuk be, az elterjedtebb fordító eszközökkel és könyvtárakkal
- A CI konfigurációt a `.travis.yml` fájlban adhatjuk meg

## Travis CI – GitHub integration



The screenshot displays the Travis CI web interface. At the top, a commit history shows three commits on April 12, 15, and 16, 2019. The commit on April 15, titled "Added Travis CI to build CodeCompass", is highlighted. Below this, a detailed view of the build for commit 3f52da6 is shown. The build status is "passed" with 29 tests. The build log is visible at the bottom, showing the following steps:

- 1 Worker information
- 6 Build system information
- 159 Adding APT Sources
- 817 \$ git clone --depth=50 --branch=ci https://github.com/Ericsson/CodeCompass.git

The build log also shows the output of the git clone command, indicating that the repository was successfully cloned.

## Travis CI

### ► Például:

```
os: linux
dist: xenial
language: java

stages:
 - compile
 - test
 - deploy

...

jobs:
 include:
 - stage: compile
 - mvn compile
 - stage: test
 - mvn test

...
```

## AppVeyor CI



- Folyamatos integrációs szolgáltatás
  - Integrálható a GitHub, GitLab, BitBucket, Visual Studio Team Services platformokkal
  - Nyílt forráskódú projektekhez ingyenesen használható
  - <https://www.appveyor.com/>
- Támogatja a Linux, a Windows és a macOS operációs rendszereket, több előkészített környezettel (*image*)
  - Kiemelt .NET és Visual Studio támogatás
- A CI konfigurációt a `appveyor.yml` fájlban adhatjuk meg

# Folytonos integráció és kiadás



## AppVeyor CI – webes interfész

aegis

Current build History

Core: Added M-tree (#21).

a year ago by Rónai Péter (committed by Roberto Giachetta)

Core: Added M-tree (#21).

a year ago by Rónai Péter (committed by Roberto Giachetta)

master: 64427e34

1.0.38

a year ago in 6 min 37 sec

Pull request #21 - M tree

Merge branch 'master' into M\_Tree

a year ago by Roberto Giachetta (committed by GitHub)

master: 7dda4590

1.0.37

a year ago in 6 min 49 sec

Core: Added Hilbert R-tree (#20).

a year ago by Rónai Péter (committed by Roberto Giachetta)

master: f84f884c

1.0.36

a year ago in 6 min 30 sec

Pull request #21 - M tree

Finished M-Tree

a year ago by Rónai Péter

master: 1de2f8d2

1.0.35

a year ago in 5 min 29 sec

Console

Messages 34

Tests

Artifacts

```
1 Build started
2 git clone -q --branch=master https://github.com/robertogiachetta/aegis.git C:\projects\aegis
3 git checkout -qf 64427e34f338989da19925565681efdb67e46c17
4 nuget restore src\AEGIS.sln
5 MSBuild auto-detection: using msbuild version '15.5.180.51428' from 'C:\Program Files (x86)\Microsoft Visual
 Studio\2017\Community\MSBuild\15.0\bin'.
6 Restoring NuGet package NUnit.3.6.1.
7 Restoring NuGet package Shouldly.2.8.2.
8 Restoring NuGet package StyleCop.Analyzers.1.0.0.
9 Restoring NuGet package Castle.Core.4.1.0.
10 Restoring NuGet package Moq.4.7.63.
11 Restoring NuGet package NUnit.ConsoleRunner.3.6.1.
12 Restoring NuGet package OpenCover.4.6.519.
13 Restoring NuGet package SimpleInjector.4.0.7.
14 Restoring NuGet package Microsoft.Win32.Primitives.4.3.0.
15 Restoring NuGet package System.Diagnostics.DiagnosticSource.4.3.0.
16 Adding package 'Microsoft.Win32.Primitives.4.3.0' to folder 'C:\projects\aegis\src\packages'
17 Adding package 'System.Diagnostics.DiagnosticSource.4.3.0' to folder 'C:\projects\aegis\src\packages'
18 Added package 'System.Diagnostics.DiagnosticSource.4.3.0' to folder 'C:\projects\aegis\src\packages'
```

## AppVeyor CI

### ► Például:

```
version: 1.0.{build} # Version format
image: Visual Studio 2017 # Build worker image
platform: Any CPU # Build platform
configuration: Debug # Build Configuration

Execute script before build
before_build:
 - dotnet restore src\MyProject.sln

Execute build script
build_script:
 - dotnet build src\MyProject.sln

Execute test script
test_script:
 - dotnet test src\MyProject.sln
```

## Jenkins

- Nyílt forráskódú folyamatos integrációs szolgáltatás
  - Integrálható a GitHub, GitLab, és egyéb projektvezető szolgáltatásokkal
  - Nem nyújt hoszting szolgáltatást, de más vállalkozások kínálnak (pl. CloudBees)
  - <https://jenkins.io/>
- A CI konfigurációt a Jenkinsfile adja meg, például:

```
pipeline {
 agent { docker { image 'maven:3.3.3' } }
 stages {
 stage('build') {
 steps {
 sh 'mvn compile'
 }
 }
 }
}
```





## GitHub Actions



- A GitHub viszonylag új, saját CI/CD szolgáltatása
  - Ingyenes fiókkal korlátozott mértékig (jelenleg 2000 futtatási perc / hó) ingyenesen használható, ez előfizetéssel bővíthető
  - <https://docs.github.com/en/actions>
- Linux (Ubuntu), Windows és macOS operációs rendszer alapú virtuális gépek futtatását támogatja, de használható Docker
  - A virtuális gépek a Microsoft felhőjében (Azure) futnak
  - Telepíthető saját *self-hosted* GitHub Actions Runner is
- A CI konfigurációt a `.github/workflows/` könyvtárban, YAML fájlokban adhatjuk meg
  - Shell utasítások mellett magunk vagy mások által elkészített *action*-öket is felhasználhatunk, mint komplex építő elemek

# Folytonos integráció és kiadás



## GitHub Actions

The screenshot displays the GitHub Actions interface for the repository **Ericsson / CodeCompass**. The top navigation bar includes links for Code, Issues (63), Pull requests (11), Actions (selected), Projects (1), Security, Insights, and Settings. The main header shows the selected workflow: **Merge pull request #515 from mcserep/search\_boost**, with 8 artifacts and a 'Re-run jobs' button.

The workflow jobs are listed on the left, including 'Build project' and several 'build' jobs for different database combinations (postgresql, sqlite3) on ubuntu-20.04 and ubuntu-18.04, along with 'parse' and 'tarball' jobs. The selected job, **build (postgresql, ubuntu-20.04)**, is expanded to show its steps:

- Install GoogleTest (10s)
- Configure CMake (0s)
- Run CMake (Postgresql) (3s)
- Run CMake (SQLite3) (0s)
- Build (4m 25s)**

The 'Build' step is expanded to show the terminal output, which includes commands like `Run make -j $(nproc)` and the compilation of various C++ objects.

On the right side of the image, a separate view shows the commit history for the repository, listing recent merges and their commit hashes (e.g., 12aa416, 3b716e1, c8dde33, 04826d2).

## GitHub Actions

### ► Például:

```
name: Build project
on: [push, pull_request]
jobs:
 build:
 runs-on: [ubuntu-latest]
 steps:
 - uses: actions/checkout@v4

 - name: Set up JDK 1.21
 uses: actions/setup-java@v4
 with:
 distribution: zulu # Azul Zulu OpenJDK
 java-version: 21

 - name: Build the project
 run: mvn compile

 - name: Run the unit tests
 run: mvn test
```